Mid-Sem Paper

Section-1: MCQ / MSQ

(10 x 1 Marks = 10 Marks)

1. Why is non-linearity (e.g., ReLU) important in neural networks?

   (a) It reduces the number of parameters in the network

   (b) It allows the network to approximate complex, non-linear functions

   (c) It makes training faster by reducing computation

   (d) It ensures the network does not overfit

   Correct Option: (b)

   Explanation: Without non-linearity, multiple layers collapse into a single linear transformation. Non-linear activation enables the network to learn complex mappings.

2. Which of the following correctly describes the role of the learning rate in gradient descent?

   (a) It determines the direction of the parameter update.

   (b) It controls the number of epochs during training.

   (c) It controls the magnitude of the parameter update step in the direction of the negative gradient.

   (d) It determines which parameters require gradients.

   Correct Option: (c)

   Explanation: The learning rate controls the step size of parameter updates during gradient descent. It scales the gradient, thereby determining how large a step is taken in the direction of the negative gradient at each iteration.

3. Consider the following PyTorch encoder:

```
import torch.nn as nn

class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1),
            nn.ReLU(),
```

```
            nn.Conv2d(32, 64, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1),
            nn.ReLU(),
        )

    def forward(self, x):
        return self.net(x)
```

If the input image has shape $(1, 3, 64, 64)$, what is the output shape?

(a) $(1, 128, 16, 16)$

(b) $(1, 128, 8, 8)$

(c) $(1, 64, 8, 8)$

(d) $(1, 128, 4, 4)$

Correct Option: (b)

Explanation: Using the convolution output size formula

$$o = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

with $k = 3$, $s = 2$, $p = 1$:

- Layer 1: $\left\lfloor \frac{64+2-3}{2} \right\rfloor + 1 = 32 \quad \Rightarrow$ Output: $(1, 32, 32, 32)$
- Layer 2: $\left\lfloor \frac{32+2-3}{2} \right\rfloor + 1 = 16 \quad \Rightarrow$ Output: $(1, 64, 16, 16)$
- Layer 3: $\left\lfloor \frac{16+2-3}{2} \right\rfloor + 1 = 8 \quad \Rightarrow$ Output: $(1, 128, 8, 8)$

Thus, the final output shape is $(1, 128, 8, 8)$.

4. Which of the following best explains why deep networks (many layers, moderate width) are generally preferred over wide networks (few layers, very large width) for tasks like image recognition, according to the Universal Approximation perspective?

   (a) Wide networks cannot represent non-linear functions; only deep networks can, because depth introduces non-linearity.

   (b) Both are theoretically equivalent by the Universal Approximation Theorem, but deep networks are chosen purely for computational efficiency.

   (c) Deep networks learn hierarchical representations (e.g., edges $\rightarrow$ textures $\rightarrow$ shapes $\rightarrow$ objects), requiring exponentially fewer neurons than a single wide layer to represent the same compositional function.

   (d) Deep networks always generalise better because they have fewer parameters than wide networks of equivalent expressivity.

Correct Option: (c)

Explanation: While both deep and wide networks are theoretically capable of approximating functions, deep networks are more efficient for compositional functions. They learn hierarchical representations (e.g., edges → textures → shapes → objects), allowing them to represent complex functions using significantly fewer parameters compared to a single wide layer.

5. In the U-Net architecture for image segmentation, skip connections between the encoder and decoder are used to:

   (a) Reduce the number of trainable parameters.

   (b) Preserve fine-grained spatial information lost during down-sampling.

   (c) Replace the need for any pooling layers.

   (d) Eliminate the need for a softmax output layer.

   Correct Option: (b)

   Explanation: Max-pooling in the encoder discards precise spatial details. Skip connections transfer feature maps from the encoder to the decoder at corresponding resolutions, allowing the network to recover fine-grained spatial information necessary for accurate pixel-level segmentation.

6. If the learning rate is too high, what is most likely to happen during backpropagation?

   (a) The model will converge faster to an optimal solution.

   (b) The loss function may oscillate or diverge instead of converging.

   (c) The gradients will become zero, leading to no learning.

   (d) The model will memorize the training data perfectly.

   Correct Option: (b)

   Explanation: A learning rate that is too high causes parameter updates to overshoot the minimum. As a result, the loss may oscillate or even diverge instead of converging, since updates "bounce" across the loss surface rather than moving steadily toward a minimum.

7. (MSQ) Which of the following statements about Stochastic Gradient Descent (SGD) compared to Batch Gradient Descent are true?

   (a) SGD updates model parameters using the entire dataset at each iteration.

   (b) SGD introduces noise in gradient updates, which can help escape sharp local minima and find flatter minima.

   (c) SGD always converges to the global minimum more efficiently than Batch Gradient Descent.

(d) SGD with momentum accumulates an exponentially decaying moving average of past gradients to accelerate convergence.

Correct Options: (b), (d)

Explanation: SGD updates parameters using a single sample or mini-batch, not the entire dataset. The stochasticity in SGD introduces noise in gradient updates, which helps escape sharp local minima and often leads to flatter minima that generalize better. SGD does not guarantee convergence to the global minimum. Momentum in SGD accumulates an exponentially decaying moving average of past gradients, helping accelerate convergence.

8. (MSQ) Which of the following statements about ReLU vs Sigmoid activations in deep CNNs are correct? Select ALL that apply.

   (a) ReLU can produce "dead neurons"—units that output 0 for all inputs and receive zero gradient—making them unable to recover during training.

   (b) Sigmoid saturates at both extremes (near 0 and near 1), causing near-zero gradients that worsen vanishing gradient through many layers.

   (c) ReLU's derivative is always 1 for positive inputs, which prevents gradient vanishing in the forward direction of backpropagation.

   (d) Leaky ReLU completely eliminates the dying neuron problem by giving a small negative slope (e.g., 0.01) for negative inputs.

   Correct Options: (a), (b), (c)

   Explanation: ReLU can suffer from the dying neuron problem where neurons output zero and receive no gradient. Sigmoid saturates at extreme values, leading to near-zero gradients and worsening the vanishing gradient problem. For positive inputs, ReLU has a derivative of 1, which helps preserve gradient flow. Leaky ReLU mitigates the dying neuron problem by allowing a small gradient for negative inputs, but it does not completely eliminate the issue.

9. (MSQ) Which of the following are true about training deep networks with the Sigmoid activation function in all hidden layers?

   (a) Sigmoid activations saturate at extreme values, leading to near-zero gradients (vanishing gradient problem).

   (b) The maximum value of $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ is 0.25, which causes gradient shrinkage when multiplied across many layers.

   (c) Replacing Sigmoid with ReLU completely eliminates all gradient-related training issues.

   (d) Sigmoid outputs are not zero-centred, which can slow down convergence due to zig-zagging gradients.

Correct Options: (a), (b), (d)

Explanation: Sigmoid activations saturate at extreme values, leading to near-zero gradients. Its derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$, causing gradients to shrink exponentially across layers. Additionally, sigmoid outputs are not zero-centred, which can slow convergence due to zig-zagging updates. Replacing sigmoid with ReLU helps mitigate vanishing gradients but does not eliminate all gradient-related issues.

10. (MSQ) Which of the following statements about RNNs are correct?

    (a) The weights of an RNN are different at each time step after unrolling.

    (b) The hidden state h_t depends on both current input x_t and previous hidden state h_{t-1}.

    (c) Backpropagation Through Time (BPTT) treats the unrolled RNN as a deep feed-forward network.

    (d) RNNs inherently solve long-term dependency problems without any modification.

    (e) In many-to-many RNNs, an output is produced at every time step.

    Correct Options: (b), (c), (e)

    Explanation: In RNNs, weights are shared across time steps, so they are not different after unrolling. The hidden state depends on both the current input and the previous hidden state. Backpropagation Through Time (BPTT) treats the unrolled RNN as a deep feedforward network. Standard (vanilla) RNNs suffer from vanishing gradients and do not inherently solve long-term dependency problems, while in many-to-many settings, outputs are produced at each time step.

Section-2: Short Answer Type Questions

(7 x 2 Marks = 14 Marks)

1. Why can a single-layer perceptron not compute the XOR function? How does adding a hidden layer solve this problem?

   Solution: A single-layer perceptron can only model linearly separable functions, whereas XOR is not linearly separable. By adding a hidden layer, the network can learn intermediate representations and combine them to form non-linear decision boundaries, enabling it to correctly model the XOR function.

2. Predict the exact printed output of the following code.

```
import torch

a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)

y = a * b + b ** 2
```

```
y.backward()

print("a.grad:", a.grad.item())
print("b.grad:", b.grad.item())
```

Solution: We have $y = ab + b^2$.

Computing partial derivatives:

- $\frac{\partial y}{\partial a} = b = 3.0$
- $\frac{\partial y}{\partial b} = a + 2b = 2.0 + 2(3.0) = 8.0$

Output: `a.grad: 3.0`, `b.grad: 8.0`

3. A two-layer feedforward neural network (no bias) has weight matrices:

$$W_1 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad W_2 = [1 \;\; 1].$$

No activation function is used. For input $x = [1, 1, 1]^T$ and ground truth $y = 50$, what is the MSE loss?

Solution:

Compute the forward pass:

$$h = W_1 x = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1+2+3 \\ 4+5+6 \end{bmatrix} = \begin{bmatrix} 6 \\ 15 \end{bmatrix}$$

$$\hat{y} = W_2 h = [1 \;\; 1] \begin{bmatrix} 6 \\ 15 \end{bmatrix} = 6 + 15 = 21$$

Mean Squared Error (single sample):

$$\text{MSE} = (\hat{y} - y)^2 = (21 - 50)^2 = (-29)^2 = 841.$$

Final Answer: $\boxed{841}$

4. In transfer learning, freezing early layers and fine-tuning later layers is a common practice. Explain why this works well, and discuss what could happen if all layers were fine-tuned from the beginning on a small dataset.

Solution: In deep networks, early layers learn generic, low-level features such as edges, textures, and simple patterns, which are transferable across different tasks and datasets. Later layers, however, learn task-specific features.

By freezing the early layers, we retain these useful generic representations and reduce the number of parameters to train, which is especially beneficial when the target dataset

is small. Fine-tuning only the later layers allows the model to adapt to the new task without overfitting.

If all layers are fine-tuned from the beginning on a small dataset, the model may overfit, since a large number of parameters are updated using limited data. Additionally, the useful pretrained features in early layers may be distorted or forgotten, leading to worse generalization.

5. Predict the exact output of the following code.

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(8, 16, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
)

x = torch.randn(1, 1, 28, 28)
out = model(x)
print("Shape:", out.shape)
print("Total elements:", out.numel())
```

Solution:

Tracing dimensions:

- Input: $[1, 1, 28, 28]$
- Conv2d$(1, 8, 3, \text{pad} = 1)$: preserves size $\Rightarrow [1, 8, 28, 28]$
- ReLU: unchanged
- MaxPool2d$(2, 2)$: $28/2 = 14 \Rightarrow [1, 8, 14, 14]$
- Conv2d$(8, 16, 3, \text{pad} = 1)$: preserves size $\Rightarrow [1, 16, 14, 14]$
- ReLU: unchanged
- MaxPool2d$(2, 2)$: $14/2 = 7 \Rightarrow [1, 16, 7, 7]$

Output:

```
Shape: torch.Size([1, 16, 7, 7])
Total elements: 784
```

Total elements $= 1 \times 16 \times 7 \times 7 = 784$.

6. Explain the differences between semantic segmentation, instance segmentation, and panoptic segmentation. In your answer, discuss (i) the type of output produced, (ii) how overlapping objects are handled, and (iii) one representative architecture for each with its key idea.

   Solution:

   Semantic Segmentation: Assigns a class label to every pixel in the image. All objects of the same class share the same label, so individual instances are not distinguished. *Handling overlap:* Overlapping objects of the same class are merged into a single region. *Example:* **U-Net**—uses an encoder-decoder architecture with skip connections to recover fine spatial details.

   Instance Segmentation: Identifies and segments each object instance separately, even if they belong to the same class. *Handling overlap:* Overlapping objects are assigned different instance masks. *Example:* **Mask R-CNN**—extends object detection by adding a parallel branch to predict a segmentation mask for each detected instance.

   Panoptic Segmentation: Combines semantic and instance segmentation by assigning every pixel both a class label and an instance ID (for countable objects). *Handling overlap:* Distinguishes individual instances while also labeling background (stuff) classes. *Example:* **Panoptic FPN**—integrates instance and semantic segmentation heads into a unified framework.

7. In sentiment classification using RNNs, two approaches are commonly used: (i) using only the final hidden state h_T, and (ii) aggregating all hidden states (e.g., sum or average).

   Why might using only h_T fail in some cases, even though RNNs are designed to capture sequential dependencies? Explain intuitively.

   Solution: Although RNNs theoretically encode past information in h_T, in practice they struggle with long-term dependencies due to vanishing gradients. As a result, information from earlier time steps may be weakly represented in the final hidden state. Aggregating all hidden states helps retain information across the entire sequence instead of relying only on h_T.

Section-3: Long Answer Type Questions

(4 x 4 Marks = 16 Marks)

1. Consider the following code that performs a single forward and backward pass through a tiny network. Carefully trace the computation and predict the exact printed output.

```
import torch

x = torch.tensor(2.0, requires_grad=True)
w1 = torch.tensor(3.0, requires_grad=True)
w2 = torch.tensor(-1.0, requires_grad=True)
```

```
b = torch.tensor(1.0, requires_grad=True)

# Forward pass
z1 = w1 * x + b # linear
a1 = torch.relu(z1) # activation
z2 = w2 * a1 # output layer
loss = (z2 - 5.0) ** 2 # MSE-style loss (single sample)

loss.backward()

print("loss:", loss.item())
print("w2.grad:", w2.grad.item())
print("w1.grad:", w1.grad.item())
print("b.grad:", b.grad.item())
print("x.grad:", x.grad.item())
```

(a) Compute the forward pass values (z_1, a_1, z_2, loss) and all four gradient values. Show the complete chain rule derivation for each gradient.

(b) If we change $x =$ `torch.tensor(-5.0, requires_grad=True)` and keep everything else the same, which gradients would become exactly zero and why?

(c) Suppose we call `loss.backward()` a second time without recomputing the forward pass. What happens and why?

Solution:

(a) Forward pass computation:

- $z_1 = w_1 \cdot x + b = 3.0 \times 2.0 + 1.0 = 7.0$
- $a_1 = \text{ReLU}(z_1) = \text{ReLU}(7.0) = 7.0$
- $z_2 = w_2 \cdot a_1 = (-1.0) \times 7.0 = -7.0$
- $\text{loss} = (z_2 - 5.0)^2 = (-7.0 - 5.0)^2 = (-12.0)^2 = 144.0$

Backward pass:

$$\frac{\partial \text{loss}}{\partial z_2} = 2(z_2 - 5.0) = 2(-12.0) = -24.0$$

- $\frac{\partial \text{loss}}{\partial w_2} = \frac{\partial \text{loss}}{\partial z_2} \cdot a_1 = -24.0 \times 7.0 = -168.0$
- $\frac{\partial \text{loss}}{\partial a_1} = \frac{\partial \text{loss}}{\partial z_2} \cdot w_2 = -24.0 \times (-1.0) = 24.0$
- $\frac{\partial a_1}{\partial z_1} = 1 \quad (\text{since } z_1 > 0)$
- $\frac{\partial \text{loss}}{\partial z_1} = 24.0 \times 1 = 24.0$
- $\frac{\partial \text{loss}}{\partial w_1} = \frac{\partial \text{loss}}{\partial z_1} \cdot x = 24.0 \times 2.0 = 48.0$
- $\frac{\partial \text{loss}}{\partial b} = 24.0$
- $\frac{\partial \text{loss}}{\partial x} = \frac{\partial \text{loss}}{\partial z_1} \cdot w_1 = 24.0 \times 3.0 = 72.0$

Printed output:

```
loss: 144.0
w2.grad: -168.0
w1.grad: 48.0
b.grad: 24.0
x.grad: 72.0
```

(b) For $x = -5.0$:

$$z_1 = 3 \times (-5.0) + 1.0 = -14.0 \quad \Rightarrow \quad a_1 = \text{ReLU}(-14.0) = 0$$

Since $z_1 < 0$, the ReLU derivative is 0, blocking gradient flow:

- w_1.grad $= 0$
- b.grad $= 0$
- x.grad $= 0$
- w_2.grad $= 0$

All gradients become zero due to the dying ReLU problem.

(c) Calling `loss.backward()` a second time without recomputing the forward pass raises a `RuntimeError`, because the computation graph is freed after the first backward pass. To reuse it, one must call:

```
loss.backward(retain_graph=True)
```

2. Let the Softplus activation be $\text{Softplus}(x) = \ln(1+e^x)$ and the Sigmoid be $\sigma(x) = \frac{1}{1+e^{-x}}$.

Useful formulas:

$$\frac{d}{dx}\ln(x) = \frac{1}{x}, \quad \frac{d}{dx}e^{ax} = ae^{ax}$$

(a) Show that $\frac{d}{dx}\text{Softplus}(x) = \sigma(x)$.

(b) Compute $\frac{d^2}{dx^2}\text{Softplus}(x)$ and express it in terms of $\sigma(x)$.

(c) Analyse the behaviour of the second derivative as $x \to +\infty$ and $x \to -\infty$, and discuss implications for gradient flow.

Solution:

(i) Let $u(x) = 1 + e^x$. Then:

$$\frac{d}{dx}\ln(u) = \frac{1}{u} \cdot \frac{du}{dx} = \frac{e^x}{1+e^x}$$

Dividing numerator and denominator by e^x:

$$\frac{1}{1+e^{-x}} = \sigma(x)$$

(ii) Since $\frac{d}{dx}\text{Softplus}(x) = \sigma(x)$:

$$\frac{d^2}{dx^2}\text{Softplus}(x) = \frac{d}{dx}\sigma(x) = \sigma(x)\big(1-\sigma(x)\big) = \frac{e^x}{(1+e^x)^2}$$

(iii) As $x \to +\infty$, $\sigma(x) \to 1$, so $\sigma(x)(1-\sigma(x)) \to 0$. As $x \to -\infty$, $\sigma(x) \to 0$, so $\sigma(x)(1-\sigma(x)) \to 0$.

The second derivative peaks near $x = 0$ (maximum value 0.25) and vanishes at extremes. This indicates that Softplus avoids sharp curvature and reduces the risk of exploding gradients. Additionally, for very negative x, the gradient remains small but non-zero (unlike ReLU), helping maintain gradient flow.

3. A student builds a CNN for 28×28 images:

```
self.net = nn.Sequential(
    nn.Conv2d(1, 32, 5, stride=1),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(32, 64, 5, stride=1),
    nn.MaxPool2d(2, 2)
)
self.fc = nn.Linear(64 * 7 * 7, 10)
```

   (a) Trace the dimensions and determine the actual shape of the output after the second `MaxPool2d`.

   (b) Will the code crash at `self.fc`? If yes, what is the correct input feature size for `nn.Linear`?

   (c) Calculate the number of parameters in the `self.fc` layer only.

Solution:

(a)

$$28 \xrightarrow{\text{Conv}(5,s=1)} 24 \xrightarrow{\text{Pool}(2)} 12 \xrightarrow{\text{Conv}(5,s=1)} 8 \xrightarrow{\text{Pool}(2)} 4$$

Final shape: $(B, 64, 4, 4)$.

(b) Yes, the code will crash. The correct input feature size for `nn.Linear` is:

$$64 \times 4 \times 4 = 1024.$$

(c) Number of parameters in `self.fc`:

$$(1024 \text{ weights} + 1 \text{ bias}) \times 10 = 10{,}250.$$

4. Consider a standard RNN where the hidden state at time t is defined as:

$$h_t = \tanh(Wh_{t-1} + Ux_t).$$

When performing Backpropagation Through Time (BPTT) to update the weight matrix W, we must compute the gradient of the loss at the final step (J_T) with respect to the weights used at the first step ($t = 1$).

(a) Write the expression for $\frac{\partial h_T}{\partial h_1}$ as a product of partial derivatives.

(b) Based on the properties of the tanh function and repeated multiplication of W, explain mathematically why the network fails to preserve long-term dependencies. What condition on W and tanh causes the gradient to vanish as T becomes large?

To address this issue, consider an LSTM with cell state C_t and forget gate f_t, with update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t.$$

(a) In a standard RNN, gradients pass through tanh or σ at every time step. Explain how the forget gate f_t in LSTM allows gradients to bypass repeated squashing.

(b) Suppose information at $t = 1$ is required for prediction at $t = 100$. What value should f_t maintain for $t = 2$ to 100 to preserve this information? Explain how this prevents vanishing gradients.

Solution:

(a) Chain rule expansion: Using Backpropagation Through Time, the gradient can be written as a product:

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}.$$

For each term:

$$\frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}\big(1 - \tanh^2(z_t)\big)\, W, \quad \text{where } z_t = Wh_{t-1} + Ux_t.$$

(b) Vanishing gradient: The derivative of tanh satisfies:

$$0 \leq 1 - \tanh^2(z) \leq 1.$$

Thus, each term $\frac{\partial h_t}{\partial h_{t-1}}$ has magnitude $\leq \|W\|$.

If $\|W\| < 1$, repeated multiplication yields:

$$\left\| \frac{\partial h_T}{\partial h_1} \right\| \approx \|W\|^{T-1} \to 0 \quad \text{as } T \to \infty.$$

Hence, gradients decay exponentially, making it difficult to preserve long-term dependencies (vanishing gradient problem).

Now consider the LSTM update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t.$$

(c) Gate mechanism: The gradient through the cell state is:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t.$$

Unlike RNNs (which involve repeated multiplication by W and $\tanh'$), the LSTM uses an additive update. This allows gradients to flow directly through the cell state without repeated squashing, forming a *constant error path*.

(d) Long-term memory condition: To preserve information from $t = 1$ to $t = 100$, the forget gate should satisfy:

$$f_t \approx 1 \quad \text{for } t = 2, \ldots, 100.$$

Then:

$$\frac{\partial C_{100}}{\partial C_1} = \prod_{t=2}^{100} f_t \approx 1.$$

Thus, the gradient remains stable and does not vanish, allowing long-term dependencies to be preserved.